

Building Recommender Systems for Movie Rating Prediction

In this assignment, we will build a recommender systems that predict movie ratings.

[MovieLense](#) has currently 25 million user-movie ratings. Since the entire data is too big, we use a 1 million ratings subset [MovieLens 1M](#), and we reformatted the data to make it more convenient to use.

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import time
from sklearn.model_selection import train_test_split
from scipy.sparse import coo_matrix, csr_matrix
from scipy.spatial.distance import jaccard, cosine
from pytest import approx
from scipy.sparse import csr_matrix
```

```
In [3]: MV_users = pd.read_csv('data/users.csv')
MV_movies = pd.read_csv('data/movies.csv')
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
```

```
In [4]: from collections import namedtuple
Data = namedtuple('Data', ['users', 'movies', 'train', 'test'])
data = Data(MV_users, MV_movies, train, test)
```

Starter codes

Now, we will be building a recommender system which has various techniques to predict ratings. The `class RecSys` has baseline prediction methods (such as predicting everything to 3 or to average rating of each user) and other utility functions. `class ContentBased` and `class Collaborative` inherit `class RecSys` and further add methods calculating item-item similarity matrix. You will be completing those functions using what we learned about content-based filtering and collaborative filtering.

`RecSys`'s `rating_matrix` method converts the (user id, movie id, rating) triplet from the train data (train data's ratings are known) into a utility matrix for 6040 users and 3883 movies.

Here, we create the utility matrix as a dense matrix (`numpy.array`) format for convenience. But in a real world data where hundreds of millions of users and items may exist, we won't be able to create the utility matrix in a dense matrix format (For those who are curious why, try measuring the dense matrix self.Mr using `.nbytes()`). In that case, we may use sparse matrix operations as much as possible and distributed file

systems and distributed computing will be needed. Fortunately, our data is small enough to fit in a laptop/pc memory. Also, we will use numpy and scipy.sparse, which allow significantly faster calculations than calculating on pandas.DataFrame object. In the `rating_matrix` method, pay attention to the index mapping as user IDs and movie IDs are not the same as array index.

```
In [36]: class RecSys():
    def __init__(self, data):
        self.data = data
        self.allusers = list(self.data.users['uID'])
        self.allmovies = list(self.data.movies['mID'])
        self.genres = list(self.data.movies.columns.drop(['mID', 'title', 'year']))
        self.mid2idx = dict(zip(self.data.movies.mID, list(range(len(self.data.movies.mID))))
        self.uid2idx = dict(zip(self.data.users.uID, list(range(len(self.data.users.uID))))
        self.Mr = self.rating_matrix()
        self.Mm = None
        self.sim = np.zeros((len(self.allmovies), len(self.allmovies)))

    def rating_matrix(self):
        """
        Convert the rating matrix to numpy array of shape (#allusers, #allmovies)
        """
        ind_movie = [self.mid2idx[x] for x in self.data.train.mID]
        ind_user = [self.uid2idx[x] for x in self.data.train.uID]
        rating_train = list(train.rating)
        return np.array(coo_matrix((rating_train, (ind_user, ind_movie)), shape=(len(self.allusers), len(self.allmovies))))

    def predict_everything_to_3(self):
        """
        Predict everything to 3 for the test data
        """
        # YOUR CODE HERE
        return np.full(len(self.data.test), 3.0)

    def predict_to_user_average(self):
        """
        Predict to average rating for the user.
        Returns numpy array of shape (#users,)
        """
        # YOUR CODE HERE
        # compute average rating for each user
        user_avg = np.true_divide(
            self.Mr.sum(axis=1),
            (self.Mr != 0).sum(axis=1)
        )

        user_avg[np.isnan(user_avg)] = 3 # default to 3 if no ratings

        # map test user IDs to indices
        test_user_idx = [self.uid2idx[uid] for uid in self.data.test.uID]

        return user_avg[test_user_idx]
```

```

def predict_from_sim(self, uid, mid):
    """
    Predict a user rating on a movie given userID and movieID
    """
    # YOUR CODE HERE
    uidx = self.uid2idx[uid]
    midx = self.mid2idx[mid]

    # user's ratings
    user_ratings = self.Mr[uidx]

    # similarities between the target movie and all others
    sim_score = self.sim[midx]

    # consider only rated movies
    rated = user_ratings > 0
    sims = sim_score[rated]
    ratings = user_ratings[rated]

    if sims.sum() == 0:
        return np.mean(user_ratings[user_ratings > 0]) if user_ratings[u

    return np.dot(sims, ratings) / sims.sum()

def predict(self):
    """
    Predict ratings in the test data. Returns predicted rating in a numpy
    """
    # YOUR CODE HERE
    predictions = []
    for uid, mid in zip(self.data.test.uID, self.data.test.mID):
        predictions.append(self.predict_from_sim(uid, mid))
    return np.array(predictions)

def rmse(self, yp):
    yp[np.isnan(yp)] = 3 #In case there is nan values in prediction, it wi
    yt = np.array(self.data.test.rating)
    return np.sqrt(((yt - yp)**2).mean())

class ContentBased(RecSys):
    def __init__(self, data):
        super().__init__(data)
        self.data = data
        self.Mm = self.calc_movie_feature_matrix()

    def calc_movie_feature_matrix(self):
        """
        Create movie feature matrix in a numpy array of shape (#allmovies, #
        """
        # YOUR CODE HERE

```

```

        return self.data.movies[self.genres].values.astype(int)

def calc_item_item_similarity(self):
    """
    Create item-item similarity using Jaccard similarity
    """
    # YOUR CODE HERE
    num_movies = len(self.allmovies)
    sim = np.zeros((num_movies, num_movies))

    for i in range(num_movies):
        for j in range(i, num_movies):
            set_i = set(np.where(self.Mm[i])[0])
            set_j = set(np.where(self.Mm[j])[0])
            intersection = len(set_i & set_j)
            union = len(set_i | set_j)
            sim[i,j] = sim[j,i] = intersection / union if union > 0 else 0
    self.sim = sim
    # YOUR CODE HERE

class Collaborative(RecSys):
    def __init__(self, data):
        super().__init__(data)

    def calc_item_item_similarity(self, simfunction, *X):
        """
        Create item-item similarity using similarity function.
        X is an optional transformed matrix of Mr
        """
        if len(X)==0:
            self.sim = simfunction()
        else:
            self.sim = simfunction(X[0]) # *X passes in a tuple format of (X)

    def cossim(self):
        """
        Calculates item-item similarity for all pairs of items using cosine
        Returns a cosine similarity matrix of size (#all movies, #all movies)
        """
        # YOUR CODE HERE
        Mr = self.Mr
        n_users, n_items = Mr.shape

        # compute user means (ignoring zeros)
        user_means = np.true_divide(Mr.sum(axis=1), (Mr != 0).sum(axis=1))
        user_means[np.isnan(user_means)] = 0

        # mean center only nonzero entries
        Mr_centered = Mr.astype(float)
        for u in range(n_users):
            Mr_centered[u, Mr[u, :] != 0] -= user_means[u]

        # transpose so items are rows

```

```

X = Mr_centered.T # shape: (n_items, n_users)

# binary mask for overlap
mask = (Mr.T != 0).astype(float) # shape: (n_items, n_users)

# dot products and norms
dot_products = X @ X.T # (n_items, n_items)
norms = np.linalg.norm(X, axis=1)
norm_matrix = np.outer(norms, norms)

# cosine similarity
with np.errstate(divide='ignore', invalid='ignore'):
    sim = np.divide(dot_products, norm_matrix)
    sim[np.isnan(sim)] = 0

# normalize to [0, 1]
sim = (sim + 1) / 2
np.fill_diagonal(sim, 1.0)

# smallest off-diagonal → 0
sim_no_diag = sim.copy()
np.fill_diagonal(sim_no_diag, np.inf)
i, j = np.unravel_index(np.argmin(sim_no_diag), sim_no_diag.shape)
sim[i, j] = 0
sim[j, i] = 0

return sim

# YOUR CODE HERE

def jacsim(self, Xr):
    """
    Calculates item-item similarity for all pairs of items using jaccard
    Xr is the transformed rating matrix.
    """
    # YOUR CODE HERE
    # ensure binary
    Xb = csr_matrix((Xr > 0).astype(int))

    # transpose to shape (n_items, n_users)
    Xb_T = Xb.T

    # intersection
    intersection = Xb_T @ Xb

    # union
    item_counts = Xb.sum(axis=0).A1
    union = item_counts[:, None] + item_counts[None, :] - intersection.T

    # avoid division by 0
    with np.errstate(divide='ignore', invalid='ignore'):
        sim = intersection.toarray() / union
        sim[np.isnan(sim)] = 0.0

```

```

    # force diagonal to 1
    np.fill_diagonal(sim, 1.0)

    return sim

# YOUR CODE HERE

```

Q1. Baseline models [15 pts]

1a. Complete the function `predict_everything_to_3` in the class `RecSys` [5 pts]

```

In [6]: rs = RecSys(data)
        yp = rs.predict_everything_to_3()
        print(rs.rmse(yp))
        ### BEGIN HIDDEN TEST
        assert(rs.rmse(yp)==approx(1.2585510334053043, abs=1e-2))
        ### END HIDDEN TEST

```

1.2585510334053043

1b. Complete the function `predict_to_user_average` in the class `RecSys` [10 pts]

Hint: Include rated items only when averaging

```

In [7]: yp = rs.predict_to_user_average()
        print(rs.rmse(yp))
        ### BEGIN HIDDEN TEST
        assert(rs.rmse(yp)==approx(1.0352910334228647, abs=1e-2))
        ### END HIDDEN TEST

```

1.0352910334228647

Q2. Content-Based model [25 pts]

2a. Complete the function `calc_movie_feature_matrix` in the class `ContentBased` [5 pts]

```

In [8]: cb = ContentBased(data)

```

```

In [9]: assert(cb.Mm.shape==(3883, 18))

```

```

In [10]: ### BEGIN HIDDEN TEST
          assert((cb.Mm[440]==np.array([0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]))
          assert((cb.Mm[0]==np.array([0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]))

```

```
assert((cb.Mm[2336]==np.array([0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
### END HIDDEN TEST
```

2b. Complete the function `calc_item_item_similarity` in the class `ContentBased` [10 pts]

This function updates `self.sim` and does not return a value.

Some factors to think about:

1. The movie feature matrix has binary elements. Which similarity metric should be used?
2. What is the computation complexity (time complexity) on similarity calculation?
Hint: You may use functions in the `scipy.spatial.distance` module on the dense matrix, but it is quite slow (think about the time complexity). If you want to speed up, you may try using functions in the `scipy.sparse` module.

```
In [11]: cb.calc_item_item_similarity()
```

```
In [12]: ### BEGIN HIDDEN TEST
assert(cb.sim.sum()>0)
assert(np.trace(cb.sim)==3883)
### END HIDDEN TEST
```

```
In [13]: ### BEGIN HIDDEN TEST
assert(cb.sim.sum()>0)
assert(cb.sim.min()==0)
assert(cb.sim.max()==1)
### END HIDDEN TEST
```

```
In [14]: ### BEGIN HIDDEN TEST
assert(cb.sim.sum()>0)
n = len(cb.allmovies)
assert(cb.sim.shape==(n,n))
### END HIDDEN TEST
```

```
In [15]: ### BEGIN HIDDEN TEST
assert(cb.sim.sum()>0)
assert(cb.sim==cb.sim.T).all()
### END HIDDEN TEST
```

```
In [16]: ### BEGIN HIDDEN TEST
assert(cb.sim.sum()>0)
assert(cb.sim[:3,:3]==approx(np.array([[1,0.2,0.25],[0.2,1,0],[0.25,0,1]])))
### END HIDDEN TEST
```

2c. Complete the function `predict_from_sim` in the class `RecSys` [5 pts]

```
In [17]: ### BEGIN HIDDEN TEST
assert(cb.predict_from_sim(2233,440)==approx(3.2010178117048347,abs=1e-2))
assert(cb.predict_from_sim(2868,2336)==approx(3.8280907095830283,abs=1e-2))
### END HIDDEN TEST
```

2d. Complete the function predict in the class RecSys [5 pts]

After completing, run below cell to calculate rating prediction and RMSE. How much does the performance increase compared to the baseline results from above?

```
In [18]: # YOUR CODE HERE
preds = cb.predict()
rmse= cb.rmse(preds)
print("RMSE:", rmse)
```

RMSE: 1.012502820366462

```
In [19]: ### BEGIN HIDDEN TEST
assert(rmse==approx(1.0128116783754684, abs=1e-2))
### END HIDDEN TEST
```

Q3. Collaborative Filtering

3a. Complete the function cossim in the class Collaborative [10 pts]

To Do:

- 1.Impute the unrated entries in self.Mr to the user's average rating then subtract by the user mean, call this matrix X.
 - 2.Calculate cosine similarity for all item-item pairs. Don't forget to rescale the cosine similarity to be 0~1.
- You might encounter divide by zero warning (numpy will fill nan value for that entry). In that case, you can fill those with appropriate values.

Hint: Let's say a movie item has not been rated by anyone. When you calculate similarity of this vector to another, you will get $\vec{0}=[0,0,0,\dots,0]$. When you normalize this vector, you'll get divide by zero warning and it will make nan value in self.sim matrix.

Theoretically what should the similarity value for $\vec{x}_i \cdot \vec{x}_i$ when $\vec{x}_i = \vec{0}$? What about $\vec{x}_i \cdot \vec{x}_j$ when $\vec{x}_i = \vec{0}$ and $\vec{x}_j$ is any vector?

Hint: You may use `scipy.spatial.distance.cosine`, but it will be slow because its cosine function does vector-vector operation whereas you can implement matrix-matrix operation using numpy to calculate all cosines all at once (it can be 100 times faster than vector-vector operation in our data). Also pay attention to the definition. The `scipy.spatial.distance` provides distance, not similarity.

3. Run the below cell that calculate yp and RMSE.

```
In [20]: # YOUR CODE HERE
cf = Collaborative(data)
cf.calc_item_item_similarity(cf.cossim)
yp = cf.predict()
rmse = cf.rmse(yp)

print("RMSE:", rmse)
print(cf.sim[0,:3])
```

```
RMSE: 1.0263081874204125
[1.          0.48022892  0.48356793]
```

```
In [21]: ### BEGIN HIDDEN TEST
assert(cf.sim.sum()!=0.)# filtering out no-answer case
n = len(cf.allmovies)
assert(cf.sim.shape==(n,n))
### END HIDDEN TEST
```

```
In [22]: ### BEGIN HIDDEN TEST
assert(np.trace(cf.sim)==3883)
### END HIDDEN TEST
```

```
In [23]: ### BEGIN HIDDEN TEST
assert(cf.sim.sum()!=0.)
assert(cf.sim==cf.sim.T).all()
### END HIDDEN TEST
```

```
In [24]: ### BEGIN HIDDEN TEST
assert(cf.sim.min()==0)
assert(cf.sim.max()==1)
### END HIDDEN TEST
```

```
In [25]: ### BEGIN HIDDEN TEST
assert(cf.sim[0,:3]==approx([1., 0.48022892, 0.48356793],abs=1e-2))
### END HIDDEN TEST
```

```
In [26]: ### BEGIN HIDDEN TEST
assert(rmse==approx(1.0263081874204125, abs=5e-3))
### END HIDDEN TEST
```

3b. Complete the function jacsim in the class Collaborative [15 pts]

3b [15 pts] = 3b-i) [5 pts]+3b-ii) [5 pts]+ 3b-iii) [5 pts]

Function `jacsim` calculates jaccard similarity between items using collaborative filtering method. When we have a rating matrix `self.Mr`, the entries of Mr matrix are 0 to 5 (0: unrated, 1-5: rating). We are interested to see which threshold method works better when we use jaccard similarity in the collaborative filtering.

We may treat any rating 3 or above to be 1 and the negatively rated (below 3) and no-

rating as 0. Or, we may treat movies with any ratings to be 1 and ones that has no rating as 0. In this question, we will complete a function jaccsim that takes a transformed rating matrix X and calculate and returns a jaccard similarity matrix.

Let's consider these input cases for the utility matrix M_r with ratings 1-5 and 0s for no-rating.

1. $M_r \geq 3$
2. $M_r \geq 0$
3. M_r , no transform.

Things to think about:

- The cases 1 and 2 are straightforward to calculate Jaccard, but what does Jaccard mean for multicategory data?
- Time complexity: The matrix M_r is much bigger than the item feature matrix M_m , therefore it will take very long time if we calculate on dense matrix.
Hint: Use sparse matrix.
- Which method will give the best performance?

3b-i) When $M_r \geq 3$ [5 pts]

After you've implemented the jaccsim function, run the code below. If implemented correctly, you'll have RMSE below 0.99.

```
In [29]: cf = Collaborative(data)
Xr = cf.Mr>=3
t0=time.perf_counter()
cf.calc_item_item_similarity(cf.jaccsim,Xr)
t1=time.perf_counter()
time_sim = t1-t0
print('similarity calculation time',time_sim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
assert(rmse<0.99)
```

```
similarity calculation time 346.75100758300005
0.9819500134453443
```

```
In [30]: ### BEGIN HIDDEN TEST
assert(rmse<1.02) #in case a small mistake in jaccard calcuation, it will gi
### END HIDDEN TEST
```

```
In [31]: ### BEGIN HIDDEN TEST
assert(rmse==approx(0.9819058692126349, abs=5e-3))
### END HIDDEN TEST
```

```
In [32]: ### BEGIN HIDDEN TEST
assert(cf.sim[0,:3]==approx([1., 0.10952085, 0.05501618],abs=1e-2))
```

```
### END HIDDEN TEST
```

```
In [33]: ### BEGIN HIDDEN TEST
assert(time_sim<30)
### END HIDDEN TEST
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[33], line 2
      1 ### BEGIN HIDDEN TEST
----> 2 assert(time_sim<30)
      3 ### END HIDDEN TEST

AssertionError:
```

3b-ii) When $M_r \geq 1$ [5 pts]

After you've implemented the jacsim function, run the code below. If implemented correctly, you'll have RMSE below 1.0.

```
In [37]: cf = Collaborative(data)
Xr = cf.Mr>=1
t0=time.perf_counter()
cf.calc_item_item_similarity(cf.jacsim,Xr)
t1=time.perf_counter()
time_sim = t1-t0
print('similarity calculation time',time_sim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
assert(rmse<1.0)
```

```
similarity calculation time 0.6438820840000972
0.9913535741888877
```

```
In [38]: ### BEGIN HIDDEN TEST
assert(rmse<1.03) #in case a small mistake in jaccard calcuation, it will gi
### END HIDDEN TEST
```

```
In [39]: ### BEGIN HIDDEN TEST
assert(rmse==approx(0.991363571262366, abs=5e-3))
### END HIDDEN TEST
```

```
In [40]: ### BEGIN HIDDEN TEST
assert(cf.sim[0,:3]==approx([1., 0.13426737, 0.07757066],abs=1e-2))
### END HIDDEN TEST
```

```
In [41]: ### BEGIN HIDDEN TEST
assert(time_sim<30)
### END HIDDEN TEST
```

3b-iii) When M_r ; no transform [5 pts]

After you've implemented the jacsim function, run the code below. If implemented correctly, you'll have RMSE below 0.96

```
In [42]: cf = Collaborative(data)
Xr = cf.Mr.astype(int)
t0=time.perf_counter()
cf.calc_item_item_similarity(cf.jacsim,Xr)
t1=time.perf_counter()
time_sim = t1-t0
print('similarity calculation time',time_sim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
assert(rmse<0.96)
```

```
similarity calculation time 0.613654624999981
0.9913535741888877
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[42], line 11
      9 rmse = cf.rmse(yp)
     10 print(rmse)
----> 11 assert(rmse<0.96)

AssertionError:
```

```
In [43]: ### BEGIN HIDDEN TEST
assert(rmse<1.0) #in case a small mistake in jaccard calcuation, it will give
### END HIDDEN TEST
```

```
In [44]: ### BEGIN HIDDEN TEST
assert(rmse==approx(0.9509126236828654, abs=5e-3))
### END HIDDEN TEST
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[44], line 2
      1 ### BEGIN HIDDEN TEST
----> 2 assert(rmse==approx(0.9509126236828654, abs=5e-3))
      3 ### END HIDDEN TEST

AssertionError:
```

```
In [45]: ### BEGIN HIDDEN TEST
assert(cf.sim[0,:3]==approx([1., 3.03561004e-02, 1.62357186e-02],abs=1e-2))
### END HIDDEN TEST
```

```

-----
AssertionError                                Traceback (most recent call last)
Cell In[45], line 2
      1 ### BEGIN HIDDEN TEST
----> 2 assert(cf.sim[0,:3]==approx([1., 3.03561004e-02, 1.62357186e-02],abs
      3      =1e-2))
      3 ### END HIDDEN TEST

AssertionError:

```

```

In [46]: ### BEGIN HIDDEN TEST
assert(time_sim<30)
### END HIDDEN TEST

```

3.C Discussion [5 pts]

1. Summarize the methods and performances: Below is a template/example.

Method	RMSE
Baseline, $Y_p=3$	
Baseline, $Y_p = \mu_u$	
Content based, item-item	
Collaborative, cosine	
Collaborative, jaccard, $M_r \geq 3$	
Collaborative, jaccard, $M_r \geq 1$	
Collaborative, jaccard, M_r	

2. Discuss which method(s) work better than others and why.

YOUR ANSWER HERE